

Medical Laboratory System

README

Nile University · School of Biotechnology | malakallam6 / Medical-Laboratory-System | Supervisor: Mohamed Mahmoud Elsayeh

1. Project Overview

The Medical Laboratory System is a desktop application for managing the everyday operations of a clinical lab. It covers five main things: **patients**, the **medical tests** they go through, the **laboratorians** who perform those tests, the **components and supplies** consumed, and the final **test results**.

The project has two parts that work together:

- **A MySQL database** — 10 tables storing all data, with proper relationships, constraints, and a trigger that automatically updates component stock whenever a new test result is recorded.
- **A Python desktop app** — a Tkinter GUI with six screens (Patients, Laboratorians, Components, Medical Tests, Test Results, Reports) so lab staff can work without writing any SQL.

A few things worth knowing:

- The database is fully normalised — no redundant data stored.
- Phone numbers live in their own tables so a person can have more than one.
- Patient age is never stored — always calculated live from the date of birth.
- Deleting a patient or laboratorian automatically removes their phone records too.
- Inventory updates itself when a test result is added — no manual stock editing needed.

2. Repository Structure

The repository has two folders and two submitted documents at the root:

```
Medical-Laboratory-System/  
|  
+-- My SQL/ <- everything database-related  
| +-- schema.sql <- CREATE TABLE + ALTER TABLE (foreign keys)  
| +-- dml.sql <- INSERT statements (all seed data)  
| +-- trigger.sql <- stock-decrement trigger
```

```
| +-- index.sql <- composite index on COMPONENT
| +-- queries.sql <- the two report queries
|
+-- src/ <- Python application
| +-- db.py <- database connection (edit this first)
| +-- styles.py <- shared colours and widget builders
| +-- main.py <- run this to start the app
| +-- patients.py
| +-- laboratorians.py
| +-- components.py
| +-- tests.py
| +-- results.py
| +-- reports.py
|
+-- Medical Laboratory System PowerPoint.pptx
+-- Medical_Laboratory_System.pdf
```

My SQL/	All SQL files. Run these once and the database is ready. schema.sql builds the tables; dml.sql fills them with sample data.
src/db.py	The only file you need to edit before running the app — add your MySQL username and password here.
src/main.py	The entry point. Run this with Python to open the dashboard.
src/styles.py	Colours and widget builders shared across all screens. You won't need to touch this.
src/*.py (rest)	One file per screen. Each handles its own queries, form inputs, and table display.
.pptx / .pdf	Project presentation and full design report.

3. How to Run

Two things to set up: the database first, then the app. Follow the steps below.

Step 1 — Load the database

Make sure MySQL is running, then open a terminal and connect:

```
mysql -u root -p
```

Run the SQL files in order:

```
SOURCE /path/to/My SQL/schema.sql;
SOURCE /path/to/My SQL/dml.sql;
SOURCE /path/to/My SQL/trigger.sql;
SOURCE /path/to/My SQL/index.sql;
```

This creates a database called **PROJECT** with 10 tables and fills them with sample data — 10 laboratorians, 10 patients, 10 medical tests, and 13 test results. Quick check:

```
USE PROJECT;
SHOW TABLES; -- should list 10 tables
SELECT COUNT(*) FROM test_result; -- should return 13
```

Step 2 — Point the app at your database

Open **src/db.py** and fill in your MySQL credentials:

```
def get_connection():
    return mysql.connector.connect(
        host = 'localhost',
        user = 'root', # your MySQL username
        password = 'your_password', # your MySQL password
        database = 'PROJECT'
    )
```

Step 3 — Install the one required package

```
pip install mysql-connector-python
```

Tkinter is included with Python on Windows and macOS. On Ubuntu/Debian: `sudo apt install python3-tk`

Step 4 — Start the app

```
cd Medical-Laboratory-System/src
python main.py
```

The dashboard opens with six tiles — one per module. The **Components** screen automatically highlights any item below its minimum stock level in red. Adding a new Test Result will silently update component quantities in the background via the database trigger.